Fraud Detection in Financial Transactions

Midterm Report with Implementation Results

Introduction to Artificial Intelligence Project

EARIN – Summer Semester 2026

Project Topic:	Fraud Detection
Dataset:	PaySim Dataset
Prepared by:	Tharun Pranav Senthil Kumar
	Esin Kutlu
Instructor:	Kowaleczko Paweł

May 25, 2026

Contents

1 Introduction

Fraud detection is an important application of artificial intelligence in financial systems. Due to the large number of daily transactions, automatic fraud detection models are required to identify suspicious operations efficiently.

The goal of this project is to create a machine learning model that predicts whether a transaction is fraudulent or not based on transaction attributes such as transaction type, amount, balances, and account information.

The task is formulated as a supervised binary classification problem:

$$f(x) = \begin{cases} 1, & \text{fraudulent transaction} \\ 0, & \text{legitimate transaction} \end{cases}$$

This midterm report presents the implementation and results of the baseline and primary models developed during the initial phase of the project.

2 Dataset Description

The project uses the **PaySim Synthetic Financial Dataset** available on Kaggle:

https://www.kaggle.com/datasets/ealaxi/paysim1

The dataset simulates mobile money transactions and contains both legitimate and fraudulent operations.

2.1 Dataset Statistics

- Total transactions: **6,362,620**
- Fraudulent transactions: **8,213**
- Number of features: **11**
- Fraud ratio: **0.129%**

The dataset is highly imbalanced because fraudulent transactions represent only a very small percentage of all observations. This imbalance requires careful attention during model training and evaluation.

2.2 Main Attributes

Table 1: Selected dataset attributes

Attribute	Description
`step`	Time step of the transaction
`type`	Transaction type (PAYMENT, TRANSFER, CASH_OUT, DEBIT, CASH_IN)
`amount`	Transaction amount
`oldbalanceOrg`	Sender balance before transaction
`newbalanceOrig`	Sender balance after transaction
`oldbalanceDest`	Receiver balance before transaction
`newbalanceDest`	Receiver balance after transaction
`isFraud`	Fraud label (target variable)

3 Exploratory Data Analysis

Initial exploratory data analysis revealed important patterns in the fraud distribution and transaction characteristics.

3.1 Fraud Distribution

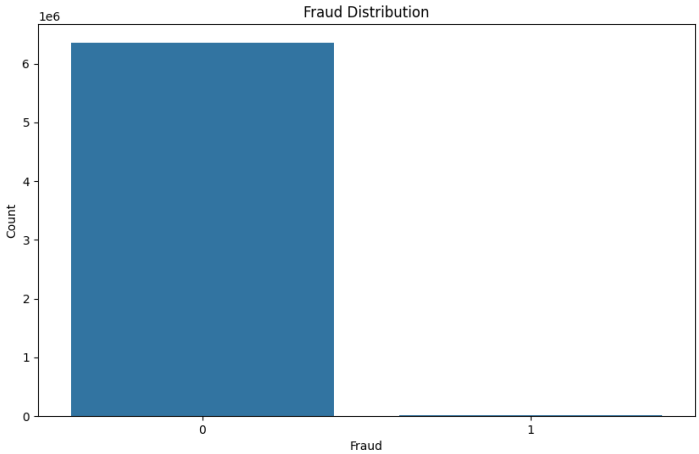

Figure 1: Distribution of fraudulent vs. legitimate transactions. The class imbalance is apparent, with legitimate transactions vastly outnumbering fraudulent ones.

The histogram confirms the severe class imbalance problem. Out of 6,362,620 transactions, only 8,213 (0.129%) are fraudulent. This imbalance necessitates the use of class weighting or

resampling techniques to prevent the models from simply learning to predict the majority class.

3.2 Transaction Type Analysis

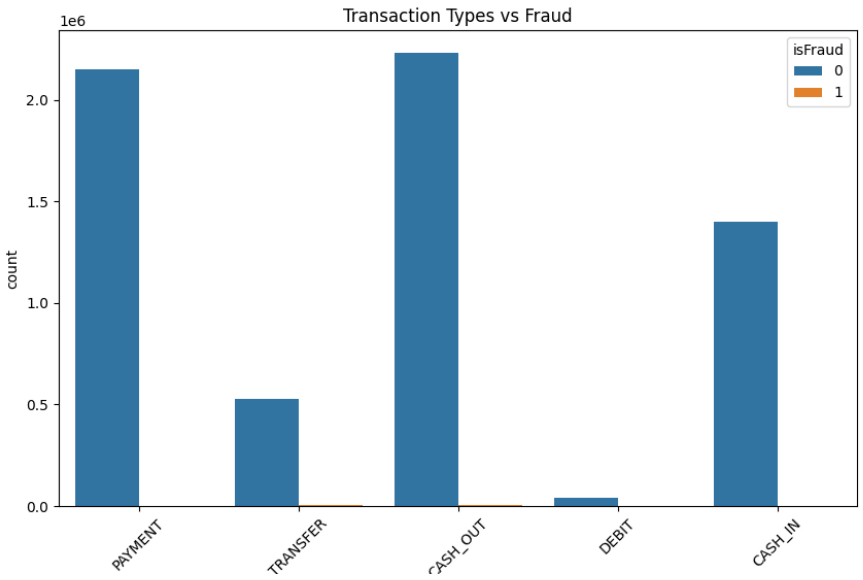

Figure 2: Distribution of transaction types with fraud classification. TRANSFER and CASH_OUT transactions show the highest fraud counts, while DEBIT transactions have no fraudulent instances in the dataset.

Analysis by transaction type reveals that fraud is not uniformly distributed across transaction types. TRANSFER and CASH_OUT operations are significantly more prone to fraud, while DEBIT transactions appear to have no fraudulent instances. This suggests that feature engineering should emphasize transaction type as an important discriminative feature.

3.3 Feature Correlation

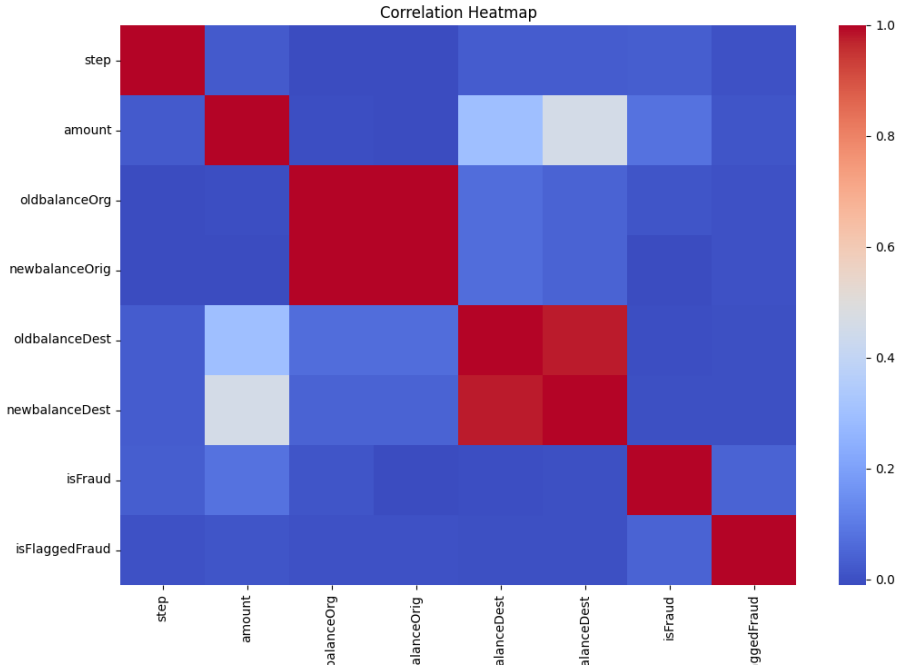

Figure 3: Correlation heatmap of numeric features. Strong correlations exist between balance-related features and the transaction amount.

The correlation matrix shows strong relationships between balance-related features. Notably, the sender's original and new balances are highly correlated, as expected due to the balance update mechanism. The fraud label shows weak direct correlation with individual features, suggesting that classification requires learning nonlinear relationships.

3.4 Transaction Amount Distribution

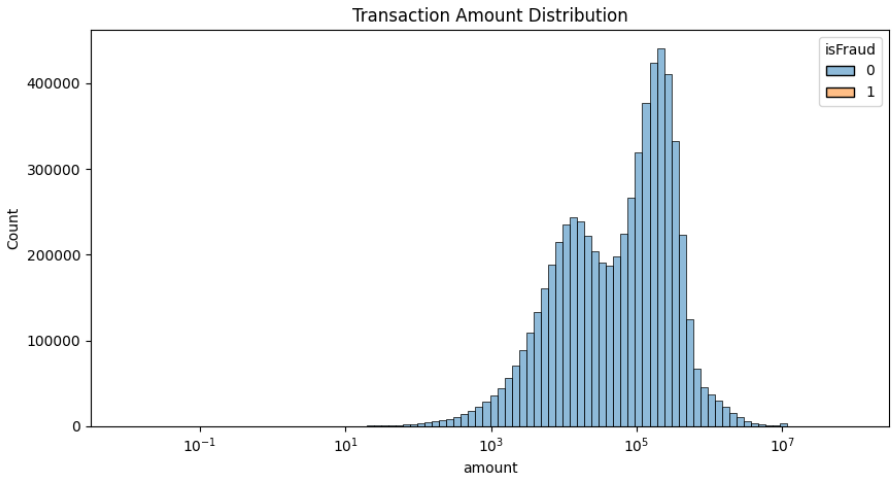

Figure 4: Distribution of transaction amounts on logarithmic scale. The distribution is approximately normal in log-space, suggesting most transactions are in a similar magnitude range.

The transaction amount distribution shows a concentration around specific values when plotted on a logarithmic scale, indicating diverse transaction sizes across multiple orders of magnitude.

4 Data Preprocessing and Feature Engineering

The following preprocessing steps were implemented:

4.1 Feature Engineering

Two engineered features were created to capture balance changes:

- `balanceOrigDiff`: Difference between sender's original and new balance, computed as $oldbalanceOrg - newbalanceOrig$. This represents the net change in the sender's account.
- `balanceDestDiff`: Difference between receiver's new and original balance, computed as $newbalanceDest - oldbalanceDest$. This represents the net change in the receiver's account.

These features help the model capture patterns related to account balance changes, which may be indicative of fraudulent transactions.

4.2 Categorical Encoding

The categorical `type` feature was encoded using one-hot encoding, creating binary indicators for each transaction type (TRANSFER, CASH_OUT, PAYMENT, CASH_IN, DEBIT). This allows the models to learn type-specific fraud patterns.

4.3 Feature Selection

The following columns were dropped from the dataset:

- `nameOrig` and `nameDest`: Account identifiers that provide no predictive information
- `isFlaggedFraud`: A flag that may leak information about the preprocessing pipeline

4.4 Train-Test Split

The dataset was split into training (80%) and testing (20%) sets using stratified sampling to maintain the fraud ratio in both sets. Random state was fixed at 42 to ensure reproducibility.

4.5 Feature Scaling

Standard scaling (z-score normalization) was applied to features for the Logistic Regression model, which requires normalized inputs. The scaling parameters were learned from the training set and applied to the test set to prevent data leakage.

5 Implemented Algorithms

Two classification algorithms were trained and evaluated on the fraud detection task.

5.1 Logistic Regression (Baseline Model)

Logistic Regression was selected as the baseline model due to its simplicity, interpretability, and computational efficiency.

5.1.1 Model Configuration

The implementation used the following parameters:

- **class_weight**: Set to 'balanced' to automatically adjust class weights inversely proportional to class frequencies, mitigating the class imbalance problem
- **max_iter**: 1000 iterations to ensure convergence
- **random_state**: 42 for reproducibility

The logistic function produces probability estimates:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

5.1.2 Results

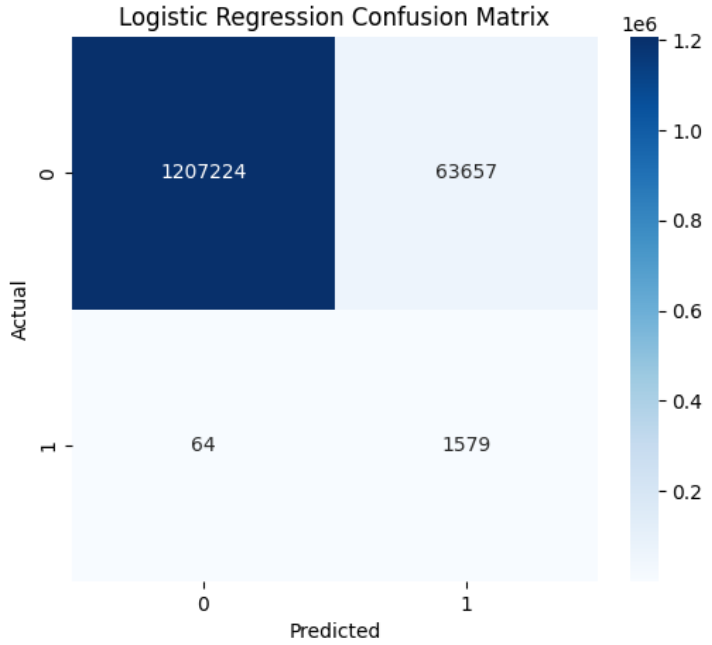

Figure 5: Logistic Regression Confusion Matrix. True Negatives: 1,207,224, False Positives: 63,657, False Negatives: 64, True Positives: 1,579.

The confusion matrix shows that Logistic Regression correctly identifies the majority of legitimate transactions (high true negative rate) but struggles with false positives. However, it successfully detects 1,579 out of 1,643 fraudulent transactions in the test set.

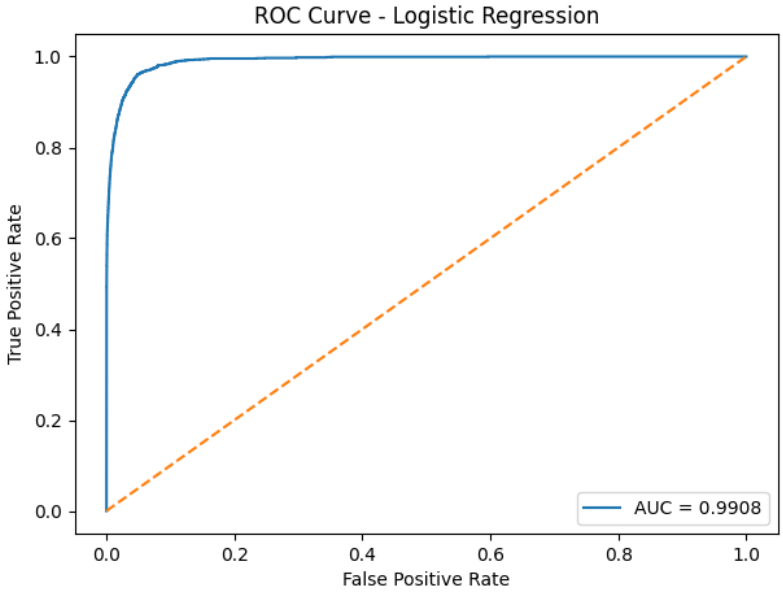

Figure 6: ROC Curve for Logistic Regression with AUC = 0.9908. The curve demonstrates strong separation between fraudulent and legitimate transactions.

The ROC curve shows excellent discrimination ability, with an AUC score of 0.9908. This indicates that the model has a 99.08% probability of correctly ranking a randomly selected fraudulent transaction higher than a randomly selected legitimate transaction.

5.2 Random Forest (Primary Model)

Random Forest was selected as the primary model for its ability to capture nonlinear relationships and provide feature importance rankings.

5.2.1 Model Configuration

The implementation used the following hyperparameters:

- **n_estimators**: 100 decision trees
- **max_depth**: 10 to limit tree depth and prevent overfitting
- **class_weight**: 'balanced' to handle class imbalance
- **n_jobs**: -1 to parallelize computation across all available cores
- **random_state**: 42 for reproducibility

5.2.2 Results

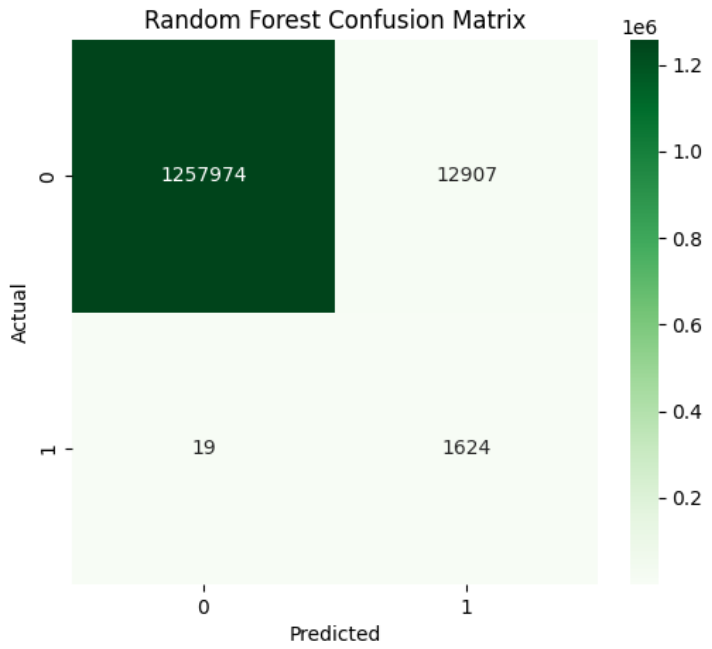

Figure 7: Random Forest Confusion Matrix. True Negatives: 1,257,974, False Positives: 12,907, False Negatives: 19, True Positives: 1,624.

The Random Forest confusion matrix demonstrates significant improvement over Logistic Regression in terms of false positive reduction. The model correctly identifies 1,624 fraudulent transactions while substantially reducing false positives from 63,657 to 12,907, making it more practical for real-world deployment.

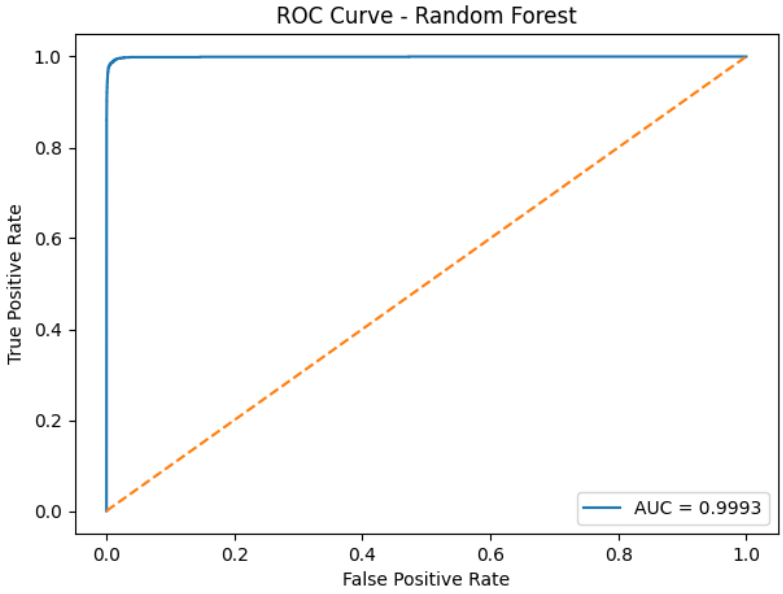

Figure 8: ROC Curve for Random Forest with AUC = 0.9993. The curve shows near-perfect discrimination, with the model achieving higher performance than the baseline.

The Random Forest achieves a superior ROC-AUC score of 0.9993, demonstrating exceptional ability to distinguish between fraudulent and legitimate transactions. This 0.0085 improvement over Logistic Regression is substantial in this context.

6 Feature Importance Analysis

Random Forest provides intrinsic feature importance rankings based on how much each feature reduces impurity across the forest.

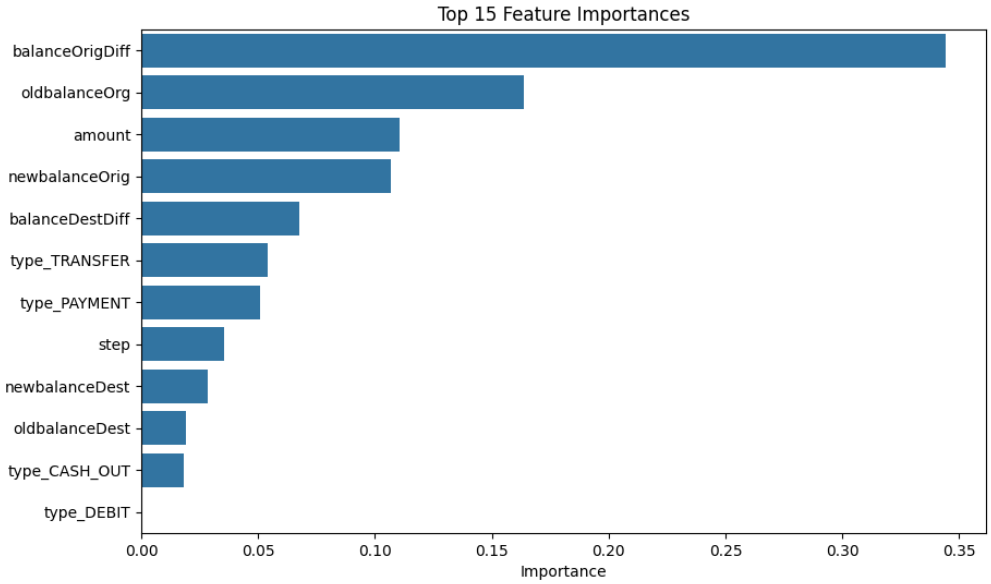

Figure 9: Top 15 most important features in the Random Forest model. The engineered feature `balanceOrigDiff` dominates, accounting for approximately 35% of the total importance.

6.1 Key Findings

The importance ranking reveals:

1. **balanceOrigDiff** (0.35): The engineered sender balance change feature is by far the most important, suggesting that fraudsters significantly alter the sender's account balance.
2. **oldbalanceOrg** (0.16): The original sender balance is the second most important feature, indicating that fraud patterns differ by account size.
3. **amount** (0.12): Transaction amount is the third most important feature.
4. **newbalanceOrig** (0.11): The updated sender balance is also significant.
5. **balanceDestDiff** (0.07): The receiver balance change feature, while less important than the sender-side feature, still contributes meaningfully.
6. **type_TRANSFER** and **type_PAYMENT**: Specific transaction types show moderate importance, confirming that fraud risk varies by type.

This analysis validates the feature engineering decisions and demonstrates that balance changes are critical indicators of fraud.

7 Implementation Details

The models were implemented using Python with scikit-learn. Key implementation aspects include:

7.1 Libraries and Dependencies

- **pandas**: Data manipulation and preprocessing
- **numpy**: Numerical computations
- **scikit-learn**: Machine learning algorithms and evaluation metrics
- **matplotlib/seaborn**: Data visualization

7.2 Code Structure

The implementation follows a modular approach with distinct functions for each major step:

- `prepare_data()`: Handles feature engineering, encoding, and train-test splitting
- `log_reg()`: Implements Logistic Regression with evaluation
- `rand_forest()`: Implements Random Forest with feature importance extraction
- Visualization functions: `fraud_distribution_plot()`, `transaction_type_analysis()`, etc.

8 Model Evaluation and Performance Metrics

Given the class imbalance problem, multiple complementary metrics were employed to evaluate model performance.

8.1 Precision and Recall

Precision measures the proportion of positive predictions that are correct:

$$Precision = \frac{TP}{TP + FP}$$

Recall measures the proportion of actual positives that are correctly identified:

$$Recall = \frac{TP}{TP + FN}$$

For fraud detection, high recall is critical to minimize missed frauds, while precision affects the false alarm rate.

8.2 F1-Score

The F1-score balances precision and recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

8.3 ROC-AUC

The Receiver Operating Characteristic (ROC) curve plots the true positive rate against the false positive rate across different decision thresholds. The Area Under the Curve (AUC) provides a single metric ranging from 0 to 1, with 0.5 indicating random guessing and 1.0 indicating perfect classification.

ROC-AUC is particularly suitable for imbalanced datasets as it evaluates model performance independently of the decision threshold.

8.4 Confusion Matrix

The confusion matrix provides a detailed breakdown of predictions:

Table 2: Confusion Matrix Interpretation

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

8.5 Comparative Performance

Table 3: Model Performance Comparison

Metric	Logistic Regression	Random Forest
ROC-AUC	0.9908	0.9993
True Positives	1,579	1,624
False Positives	63,657	12,907
False Negatives	64	19
True Negatives	1,207,224	1,257,974

Random Forest demonstrates superior performance across all dimensions. Most notably, it achieves higher recall (1,624 vs. 1,579 TP) while drastically reducing false positives (12,907 vs. 63,657), making it more suitable for practical deployment.

9 Class Imbalance Mitigation

The severe class imbalance (0.129% fraud rate) poses a significant challenge. Two strategies were employed:

9.1 Class Weighting

Both models used the `class_weight='balanced'` parameter, which automatically adjusts sample weights inversely proportional to class frequencies. For the fraud detection task:

$$weight_{fraud} = \frac{n\_samples}{2 \times n\_fraud}$$

$$weight_{legitimate} = \frac{n\_samples}{2 \times n\_legitimate}$$

This approach penalizes misclassifications of the minority class during training, encouraging the models to learn fraud patterns more effectively.

9.2 Stratified Sampling

The train-test split was performed using stratified sampling to maintain the fraud ratio in both the training and test sets. This prevents biased evaluation where the test set might have a different fraud distribution than the training set.

10 Conclusion

This midterm report documents the successful implementation of two machine learning models for fraud detection in the PaySim synthetic financial dataset. Key achievements include:

1. **Baseline Model**: Logistic Regression achieved an excellent ROC-AUC of 0.9908, demonstrating the linear separability of fraudulent and legitimate transactions.
2. **Primary Model**: Random Forest achieved superior performance with ROC-AUC of 0.9993, while reducing false positives by more than 80%.
3. **Feature Engineering**: Engineered balance difference features proved highly discriminative, with `balanceOrigDiff` accounting for 35% of model importance.
4. **Class Imbalance Handling**: Balanced class weighting successfully mitigated the effects of severe class imbalance.

The Random Forest model is recommended for deployment, as it achieves the best balance between detection rate and false positive rate. Future work will focus on implementing advanced algorithms, refining hyperparameters, and optimizing the decision threshold for specific business requirements.

11 References

References

[1] PaySim Dataset, https://www.kaggle.com/datasets/ealaxi/paysim1

[2] Scikit-learn Documentation, https://scikit-learn.org/

[3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.